МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

КУРСОВАЯ РАБОТА

по дисциплине «Программирование»

Тема: Обработка изображений

Студент гр. 1383 _____ Малимон Г.Д

Преподаватель _____ Кринкин К.В.

Санкт-Петербург
2022

ЗАДАНИЕ
НА КУРСОВУЮ РАБОТУ

Студент Малимон Г.Д

Группа 1383

Тема работы : обработка изображения

Исходные данные:
Изображение в формате bmp.

Содержание пояснительной записки:
«Содержание», «Введение», «Заключение», «Список использованных источников»

Дата выдачи задания: 22.03.2022

Дата сдачи реферата: 03.06.2022

Дата защиты реферата: 04.06.2022

Студент _____ Малимон Г.Д

Преподаватель _____ Кринкин К.В.

АННОТАЦИЯ

Программа с CLI интерфейсом обрабатывает изображение несколькими способами:

1. фильтр RGB-компонент.
2. Смена одного цвета на дугой по выбору.
3. Нарезка изображения на N*M частей.

СОДЕРЖАНИЕ

	Введение	5
1.	Создание программы	6
1.1.	Считывание аргументов	6
1.2.	Обработка аргументов	7
1.3.	Функции считывания и записи изображения	8
1.4.	Фильтр RGB -компонент	9
1.5.	Смена цвета	9
1.6.	Нарезка на части	9
3.	инструкция	9
2.	Тестирование	10
2.1.	Заключение	10
2.2.	Список использованных источников	11

ВВЕДЕНИЕ

Целью данной работы является создание программы, способной считывать, записывать и обрабатывать изображение.

Для достижения поставленной цели требуется решить следующие задачи:

1. Создние программы
    1.1. Создать интерфейс программы
    1.2. Создать функции обработки изображения, соответствующие выше описанным действиям
2. Тестирование

1. СОЗДАНИЕ ПРОГРАММЫ

1.1. Считывание аргументов

Взаимодействие с программой происходит через передачу аргументов(ключей) с запуском программы, для чего реализован getopt_long; а так же через текстовые сообщения непосредственно в ходе работы программы, например работа над ошибками ввода неправильных аргументов.

1.2. Обработка аргументов.

В функции main поочерёдно отправляются введённые getopt аргументы в switch на дальнейшую работу обработку.

Есть ключи первого порядка:

“-p” или «—path», говорит что далее вы введёте путь по которому в дальнейшем будут сохраняться итоговые файлы выхода программы.

”-n” или «—name», говорит что далее вы введёте имя, которым будет называться файл с итоговым изображением.

Данные ключи должны поступать впереди всего так как по ним запускаются функции считывания и обработки имени и директории, вкратце они записывают имя или директорию в динамический массив (так как и то и то может быть слишком длинными для стека), а потом проверяют на валидность, или корректирует и сохраняют.

Ключи второго порядка это сами функции программы:

-R -G -B: (подзадание ргб компонент) эти ключи требуют аргумента а именно числа в которое надо изменить нужную компоненту изображения, стоит добавить, что далее программа проверит это число и предложит ввести заново если засечёт ошибку ввода.

-c:(change color) тут нет аргумента так как ключ ведёт к подзаданию изменения одного цвета на другой, в ходе выполнения этого маршрута,

программа запросит поочерёдно 2 цвета либо в формате (r, g, b) либо выбором готового по ключу, если на этапе выбора цвета пользователь допускает ошибку в вводе цвета или или ключа к цвету, программа сообщит об ошибке программа предложит ввести заново нудный цвет.

-d: (devide_image) требует аргумент ввиде двух чисел M и N означающих количество частей на которое надо разделить картинку. В ходе выполнения программа выведет ошибку если не введены числа и поставит дефолтные значения, либо если разбиене превышает нормы, просто завершится с ошибкой неверного ввода. Прошу заметить что максимальное разбиение картинки, это 625x625, так как реализованная в коде, функциия создания имён к файлам кусков изображения, работает по принципы двузначной 25ой системы, код можно доработать до большего разбиения путём увеличения длинны имён файлов, но и 625x625 это много.. очень много это 625**2 файлов, не надо даже проверять, у меня это 6 мнут обрабатывалось, не надо.

1.3. Функции считывания и записи изображения.

Функция считывания (readbmp) получает на вход указатель на структуру bmpfile и строку с путем к файлу. Открывается файл на бинарное чтение. Если на каком-то этапе оказывается ошибка из-за неверного формата файла, программы выводит об это ошиибку и завершается.

Функция записи (savebmp) работает аналогичным образом. В конце память под пиксели очищается.

1.4. фильтр RGB-компонент

функция просто изменяет значение нужной компоненты каждого пикселя, для вода самой компоненты есть функция проверки и до ввода если введено неправильное значение.

1.5. замена цвета

функция смены цвета так же как и в **1.4** взаимодействует с каждым пикселем и меняет его цвет если это нужно, для сравнения двух цветов есть отдельная функция компаратор, по завершении замены цвета, программа выводит количество изменённых пикселей.

1.6. Нарезка изображения

Можно было выбрать, нарисовать лини или нарезать на разные файлы, нарисовать линии прямо поверх картинки скучно, поэтому нарезка. Корень всего алгоритма в 2 функциях. Первая, это создатель имён к файлам, дело в том что, при создании 1000 файлов, нужно как-то индивидуально назвать каждый, для чего нужен состовитель строк так же, чтобы проверить работоспособность программы не хватит просто посмотреть на 1000 файлов со случайными кусками изображения, поэтому названия должны быть сформованы определённым образом. Создатель имен получает на вход 2 числа, а именно координаты данного кусочка изображения в минимом массиве, в данной реализации можно представить, что изображение -это двумерный массив его кусочков, таким образом индекс [I][J] каждого сектора может указвать на его индивидуальный номер, сначала берётся остаток от деления на 25 x, у прибавляется 65 и переводится в символ таблицы ascii, (так как номера английских заглавных букв это [65-90]), это гарантирует отсутствие в будущем названии неправильных знаков, когда остаток от деления равен 25, счётчик пребрасывает единицу в следующй разряд, название имени состоит из 4 букв например [ABBA] первые 2 символа AB отвечают за X величину, BA за Y величину, таким образом можно сказать, что в блочном представлении разбитой картинки, сектор с именем ABAC имеет координату [3, 1], то есть третий сверху.

Вторая главная функция — это сама нарезка, так как частички одинаковые, создаётся temp структура нужного N*M размера, фотография обрезается по размерам, чтобы каждая сторона делилась на нужное целое количество частей(потеря 2-6 пикселей не так важна, а вот смысла в наличии кусков с 1 выбитым пикселем с изображения и чёрным фоном -нету), далее цикл обхода картинки: внешний слой находит индекс верхнего левого угла каждого сектора(части) во внешнем цикле так же есть 2 переменные **NAMEX и NAMEY** с помощью этих переменных сохраняются координаты блока(да, можно вычислять координаты блока по индексам внешнего циикла, но это лишние и неприятные вычисления), внутренний цикл обходит уже каждый пиксель в пределах нужного сектора, и копирует на temp структуру, далее она и сохраняется с оригинальным именем с функцией name_construct в которую передаются **namex и namey**.

2. Инструкция по работе:

весь код находится в одном файле, поэтому запус происходит как и любая другая прростая программа.

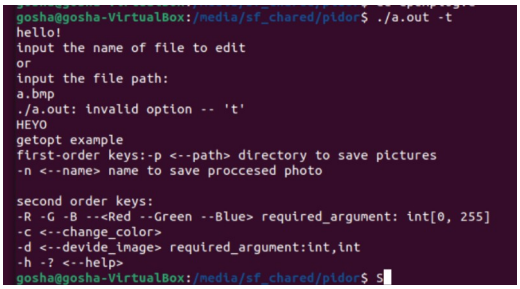
На начале стоит просто запустить файл с ключом -h, программа спросит файл для обработки, введите его название, и программа просто выведет подсказку по ключам.

В дальнейшем вы можете просто вводить нужные ключи со своими аргументами, есл чтото будет не так или нужно чтото ещё, программа спросит. Удачного пользованя!

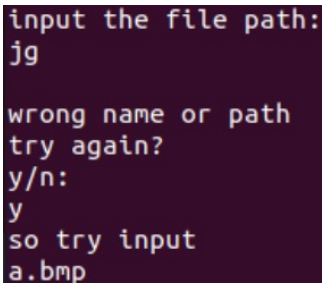
3. ТЕСТИРОВАНИЕ

2.1. ввод неверных значений:

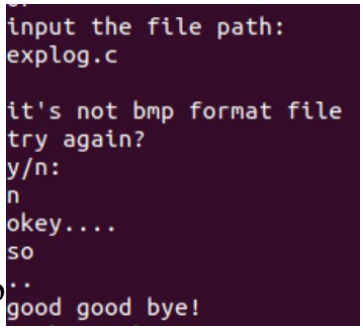
Неверный ключ при запуске:



Неверное имя файла или путь:

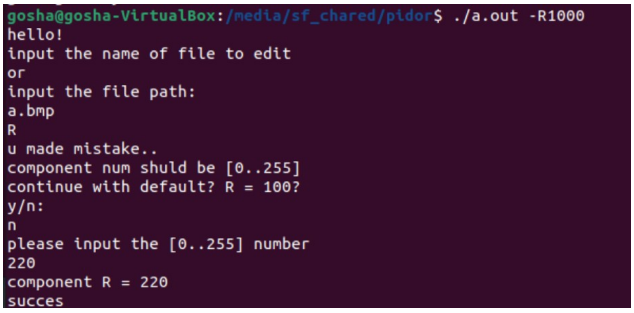


Неверный файл:



(есть ещё 4 ошибки, как например наличие сжатия)

Неправильный аргумент компоненты цвета(-R1000):



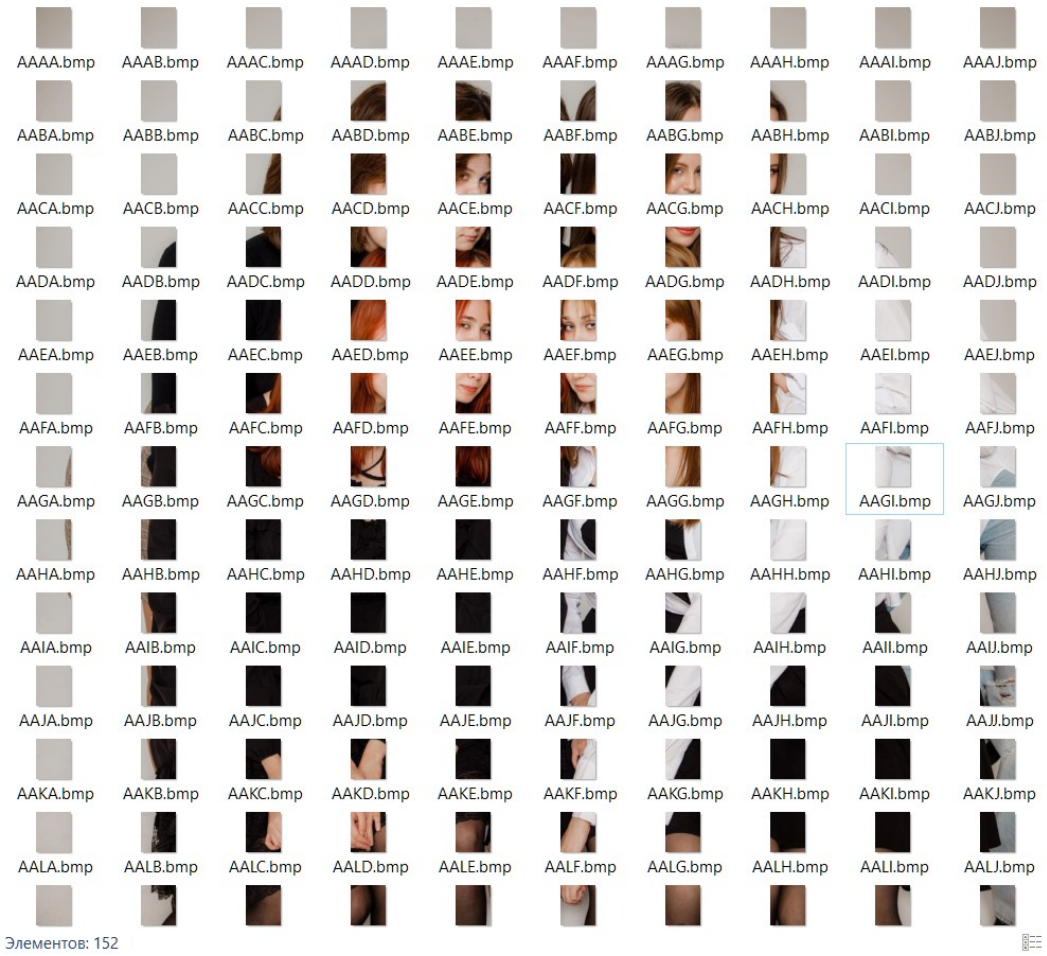
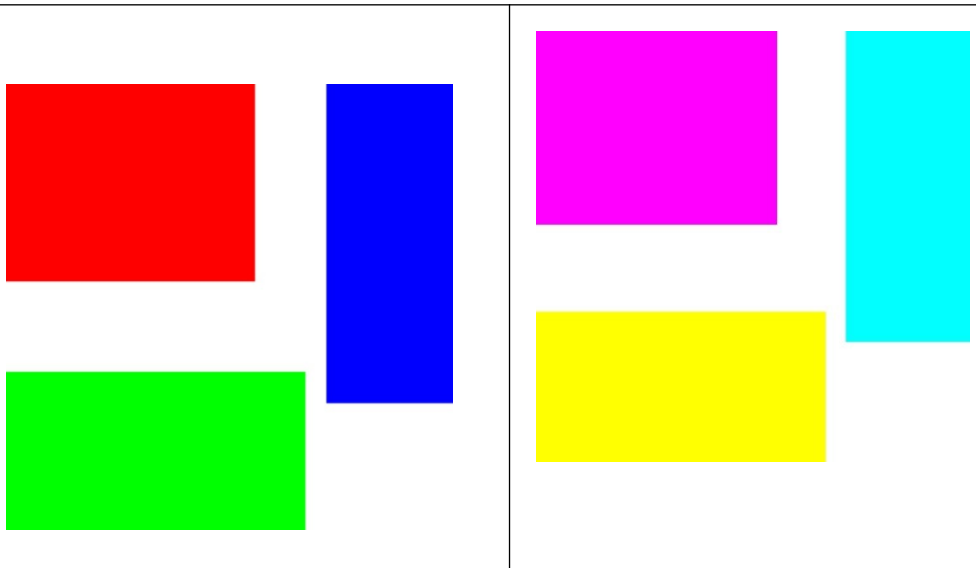
Слишком большое разбиение:

```
gosha@gosha-VirtualBox:/media/sf_chared/pidor$ ./a.out -p -d5000,5000
hello!
input the name of file to edit
or
input the file path:
a.bmp
please input path
hhh
to big parametrs for devide
sorry
gosha@gosha-VirtualBox:/media/sf_chared/pidor$
```

В основном как-то так, остальные выводы ошибок лучше на опыте увидеть.

2.2 работа функций

Функция	Оригиинал файла
	
функция	Фильтр RGB компонент
	./a.out -R0 -G0 (удаление всех компонент кроме синих оттенков)
	

функция	Разделение 15 на 10
	./a.out -p -d15,10 (разделить и сохранить в папку)
	
Функция	Замена цвета
	./a.out -c -c -c (зелёный-желтый, красный-маджента, синий-циан) слева: оригинал — справа: изображение
	

ЗАКЛЮЧЕНИЕ

Разработана программа, способная считать файл изображения «bmp», обработать и сохранить, разработаны методы обработки пиксельного массива для выполнения нужных задач по изменению цвета или вырезания частей изображения, изучена структура строения файлов типа: изображения; реализована функция getopt_long, изучена идея CLI, изучены Стандарты POSIX

.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. http://cppstudio.com *— стандартные функции языка си*

*2.*https://ziggi.org/chtenie-bmp-v-c-versiya-2/ *немного усложненная версия считывания bmp файла и теория о нём для своевременногго вывода ошибок о чтении*

3. код с лекции о работе с bmp от Чайки К.В

4. код с лекции о getopt_long от Чайки.К.В